CO2 Assignment — Brute Force and Exhaustive Search

CSA06 — Design and Analysis of Algorithms

Student	To be filled in
Registration Number	To be filled in
Assignment	Assignment 2 – CO2
Marks in uploaded assignment PDF	50
Class announcement	100 points; confirm authoritative total
Due date	October 29 (class announcement)

Starter report based on the supplied eight-page assignment. Complete Q7 using evidence from your own Q1–Q6 experiments. The report intentionally distinguishes analytical expectations from measured results.

Q1 — Sorting under different data conditions

Selection Sort performs $\Theta(n^2)$ comparisons regardless of initial order, with at most n–1 swaps. Optimized Bubble Sort can stop after a no-swap pass; worst case $\Theta(n^2)$, best case $\Theta(n)$. Insertion Sort is $\Theta(n^2)$ in reverse order and $\Theta(n)$ on already sorted input, making it a strong candidate when about 95% of records are already ordered and only small batches arrive. Validate with comparison, swap/shift and time counts.

Q2 — Sequential search

Successful search at zero-based position k needs k+1 comparisons; unsuccessful search needs n. Best-case $\Theta(1)$, average/worst $\Theta(n)$. It can be reasonable for small, frequently changing lists with infrequent lookups and no useful index. As the system expands to 20 facilities with more records and queries, measure total lookup load and consider an indexed structure.

Q3 — Brute-force string matching

For text length n and pattern length m, the algorithm tests up to n–m+1 alignments and can make $\Theta((n–m+1)m)$ character comparisons. A pattern with an early mismatch is cheap; a repeated-prefix text/pattern pair can repeat many successful comparisons before failing. Record comparison counts for three designed cases.

Q4 — Brute-force closest pair

Every unique unordered pair is examined once, so pair count is n choose 2 = n(n–1)/2. Time $\Theta(n^2)$, auxiliary space $\Theta(1)$. Use multiple point counts, record pair/distance counts, and define a response-time threshold for your environment. The key limitation is quadratic pair enumeration.

Q5 — Brute-force convex hull

For each candidate point pair, calculate the orientation/cross product for every other point. If all other points lie on one side or on the line, the pair is a supporting-edge candidate. Straightforward enumeration is $\Theta(n^3)$, with degeneracies (collinear points, duplicates) requiring careful handling. The included implementation returns unique boundary candidates ordered around their centroid; test collinear and duplicate-point cases as well as interior points.

Q6 — Exhaustive search / knapsack

Enumerate all 2^n package subsets, sum weights and values, discard over-capacity subsets, and retain the feasible subset with maximum value. Time $\Theta(n \cdot 2^n)$, memory O(n) for the current implementation's candidate tracking plus input. For small instances, independently enumerate solutions to verify the optimum. Each added package doubles the candidate subsets.

Q7 — Day-4 diagnose, redesign and validate

The official brief explicitly requires choosing a bottleneck evidenced by Q1–Q6, not simply naming a more advanced textbook algorithm. A suitable candidate is the brute-force closest-pair solution: it calculates n(n–1)/2 distances. First record a prediction, then implement a redesigned method (e.g., sweep-line or divide-and-conquer closest pair), compare distance calculations and runtime on equivalent datasets, and explain memory/preprocessing trade-offs. This starter package leaves Q7 code as a task because the redesign must be based on measured Q1–Q6 evidence.

Testing and evidence

The test harness and experiment runner are included. Primary evidence should be algorithm-specific operation counts, not timing alone. Times vary by hardware and Python environment. Capture authentic source and terminal screenshots into each Output_Screenshots directory; no screenshots are fabricated in this package.

Submission checklist

Fill in student details; execute all scripts; review raw CSV experimental data; capture screenshots; verify hull edge cases; complete Q7 after analyzing actual Q1–Q6 measurements; prepare a DOCX report if the instructor requires the named format; and confirm the marks/deadline in the LMS. The uploaded PDF says 50 marks and leaves submission date blank, while the class post says 100 points due Oct 29.